

Computergrafik Praxis - Groundtruth Dokumentation

Dieses Notebook ist sukzessiv nach Teilaufgaben aufgebaut. Jede Teilaufgabe ist einklappbar und enthaelt direkt am Ende ein eigenes Glossar mit den wichtigsten Begriffen und Abkuerzungen.

Stand: 2026-05-01

Quellen im Repository:

- `tasks/task02.md`
- `tasks/task03.md`
- `tasks/task03_dokumentation.md`
- `tasks/task04.md`
- `tasks_src/task02/task02.cpp`
- `tasks_src/task03/task03.cpp`
- `tasks_src/task04/task04.cpp`
- `assets/shaders/*.vert` und `assets/shaders/*.frag`

▼ Aufgabe 2.1 - Vertex-Daten auf die GPU laden und rendern

Ziel

In Aufgabe 2 soll das Modell `stormtrooper.obj` geladen, in einen GPU-Buffer kopiert und mit OpenGL gerendert werden. Das vorhandene Geruest in `tasks_src/task02/task02.cpp` laedt bereits Fenster, Shader, Kamera und Modell. Die fehlenden Stellen sind VAO, VBO, Drawcall und Cleanup.

Ausgangsdaten auf der CPU

Das Modell wird ueber `Model::Load("models/stormtrooper.obj")` geladen. Intern benutzt `Model::Load` den `tinyobjloader`. Aus den OBJ-Indizes entsteht eine flache Vertexliste:

```
const std::vector<Vertex>& vertices = model.GetVertices();
```

Der gemeinsame Vertex-Typ des Projekts ist:

```
struct Vertex
{
    Vec3f position;
    Vec3f normal;
    Vec3f color;
```

```
    Vec3f uv;
};
```

Fuer Aufgabe 2 liest der Shader nur `position`. Trotzdem wird der ganze `Vertex`-Struct in den VBO kopiert. Nicht gelesen werden `normal`, `color` und `uv`, weil `task02.vert` nur dieses Attribut besitzt:

```
layout (location = 0) in vec3 in_position;
```

VBO, VAO und Shader-Zusammenhang

Merksatz:

VBO = Speicher mit Vertexdaten

VAO = Anleitung, wie dieser Speicher gelesen wird

Shader = Empfaenger der gelesenen Attribute

Code fuer den GPU-Upload

```
GLuint vbo = 0;
glCreateBuffers(1, &vbo);

glNamedBufferData(
    vbo,
    model.GetVertices().size() * sizeof(Vertex),
    model.GetVertices().data(),
    GL_STATIC_DRAW);
```

Code fuer das VAO

```
GLuint vao = 0;
glCreateVertexArrays(1, &vao);

glVertexArrayVertexBuffer(vao, 0, vbo, 0, sizeof(Vertex));
glVertexArrayAttribFormat(vao, 0, 3, GL_FLOAT, GL_FALSE, 0);
glVertexArrayAttribBinding(vao, 0, 0);
glEnableVertexArrayAttrib(vao, 0);
```

Die Position steht im `Vertex` direkt am Anfang. Deshalb ist der relative Offset `0`. Der Abstand zum naechsten Vertex ist `sizeof(Vertex)`.

Rendern

Im Renderloop werden Model-, View- und Projection-Matrix gesetzt. Danach wird gezeichnet:

```
shader.Use();
glBindVertexArray(vao);

glUniformMatrix4fv(0, 1, GL_FALSE, modelMat.Data());
glUniformMatrix4fv(1, 1, GL_FALSE, viewMat.Data());
glUniformMatrix4fv(2, 1, GL_FALSE, projMat.Data());
```

```
glDrawArrays(GL_TRIANGLES, 0, (GLsizei)model.NumVertices());
```

`glDrawArrays` passt hier, weil `Model::Load` eine expandierte Dreiecksliste erzeugt. Es wird kein Indexbuffer gebraucht.

Cleanup

```
glDeleteBuffers(1, &vbo);  
glDeleteVertexArrays(1, &vao);
```

Glossar

- **CPU**: Hauptprozessor. Lädt das Modell und schickt Daten zur GPU.
- **GPU**: Grafikprozessor. Führt Shader aus und rendert das Bild.
- **Vertex**: Ein Punkt eines Meshes mit Zusatzdaten wie Position, Normale oder UV.
- **VBO**: Vertex Buffer Object. Speicher für Vertexdaten auf der GPU.
- **VAO**: Vertex Array Object. Beschreibt, wie Daten aus dem VBO als Shaderattribute gelesen werden.
- **Offset**: Byteposition eines Feldes innerhalb eines Vertex.
- **Stride**: Byteabstand von einem Vertex zum nächsten.
- **Shader**: GPU-Programm. Aufgabe 2 nutzt Vertex- und Fragmentshader.
- **Uniform**: Shaderwert, der für den ganzen Drawcall gleich bleibt, zum Beispiel eine Matrix.
- **Drawcall**: Zeichenbefehl an OpenGL, zum Beispiel `glDrawArrays`.
- **OBJ**: 3D-Modellformat.
- **tinyobjloader**: Bibliothek zum Lesen von OBJ-Dateien.

▼ Aufgabe 2.2 - Dokumentation der OpenGL-Funktionen

Ziel

Aufgabe 2.2 verlangt eine kurze, ernsthafte Dokumentation der verwendeten OpenGL-Funktionen. Wichtig ist nicht nur, was jede Funktion einzeln tut, sondern wie sie zusammenarbeiten.

Zusammenhang der Funktionen

Funktionsbedeutung

- `glCreateBuffers` : Erzeugt echte, initialisierte Buffer-Objekte.
- `glNamedBufferData` : Reserviert Speicher und kann Initialdaten direkt kopieren.
- `glNamedBufferSubData` : Kopiert Daten in einen bereits angelegten Bufferbereich.
- `glCreateVertexArrays` : Erzeugt ein VAO.
- `glVertexArrayVertexBuffer` : Verbindet einen VBO mit einem VAO-Bindungspunkt.

- `glVertexArrayAttribFormat` : Beschreibt das Format eines Attributs.
- `glVertexArrayAttribBinding` : Verbindet Attributlocation und Bindingpunkt.
- `glEnableVertexArrayAttrib` : Aktiviert ein Attribut.
- `glBindVertexArray` : Macht das VAO fuer folgende Drawcalls aktiv.
- `glDrawArrays` : Zeichnet eine nicht-indizierte Vertexliste.

glCreateBuffers vs. glGenBuffers

`glCreateBuffers` gehoert zum modernen Direct-State-Access-Stil. Es erzeugt direkt ein initialisiertes Buffer-Objekt. `glGenBuffers` stammt aus aelterem OpenGL und reserviert historisch zunaechst nur Namen; das Objekt entsteht durch spaeteres Binden an ein Target. Die Aufgabenstellung fordert deshalb `glCreateBuffers` .

Glossar

- **API**: Programmierschnittstelle, hier OpenGL.
- **OpenGL State**: Zustand, den OpenGL intern speichert, zum Beispiel aktives VAO.
- **DSA**: Direct State Access. Objekte werden direkt ueber ihre IDs veraendert.
- **Binding**: Verknuepfung eines Objekts mit einem Slot oder Zustand.
- **RenderDoc**: Grafikdebugger zum Untersuchen von Frames, Buffern und Drawcalls.
- **Resource Cleanup**: Freigeben von GPU-Objekten mit `glDelete...` .

▼ Aufgabe 3.0 - Setup

Ziel

Das aktuelle Rahmenprogramm wird frisch geklont, weil Aufgabe 3 auf einer aktualisierten Basis aufsetzt. Danach wird `tasks_src/task03/task03.cpp` gebaut und mit dem Asset-Pfad gestartet.

Wichtige Schritte

```
cmake -B build -S .
cmake --build build --target task03 --config Debug
.\build\Debug\task03.exe assets\
```

Das Programm soll nach dem Start ein Koordinatensystem bzw. spaeter die Aufgabe-3-Szene anzeigen.

Glossar

- **Clone**: Lokale Kopie eines Git-Repositories.
- **CMake**: Werkzeug, das Build-Dateien erzeugt.
- **Target**: Konkretes Bauziel, zum Beispiel `task03` .
- **Assets**: Laufzeitdaten wie Shader, Modelle und Texturen.

- **Working Directory:** Arbeitsordner, von dem relative Pfade ausgehen.

▼ Aufgabe 3.1 - Farbattribute

Ziel

Die Vertices enthalten nicht nur Position und Normale, sondern auch eine Farbe. Diese Farbe soll vom VBO ueber das VAO in den Vertexshader und von dort in den Fragmentshader gelangen.

Datenweg

Layout

- Location 0: Position
- Location 1: Normalenvektor
- Location 2: Farbe

`color` beginnt nach `position` und `normal`, also nach `6 * sizeof(float)`.

Glossar

- **Farbattribut:** Farbe, die pro Vertex gespeichert ist.
- **RGB:** Rot, Gruen, Blau.
- **Location:** Fester Slot eines Shaderattributes.
- **Interpolation:** OpenGL mischt Werte ueber ein Dreieck fuer jedes Fragment.
- **Fragment:** Kandidat fuer einen finalen Pixel.

▼ Aufgabe 3.2 - Primitive: Quader, Zylinder, Kugel

Ziel

Es werden Funktionen gebaut, die Quader, Zylinder und Kugel als Dreieckslisten erzeugen. Alle Primitive liegen zuerst lokal um den Ursprung. Groesse und Position entstehen spaeter ueber Matrizen.

Gemeinsames Prinzip

Quader

Der Quader nutzt acht Eckpunkte mit halber Ausdehnung `0.5`. Jede Flaeche besteht aus zwei Dreiecken. Insgesamt entstehen `6 * 2 * 3 = 36` Vertices. Jede Flaeche bekommt eigene Vertices, damit jede Flaeche eine eigene konstante Normale haben kann. Dadurch bleiben die Kanten hart.

Zylinder

Der Zylinder liegt entlang der Z-Achse. Die Kreisringe werden ueber Winkel erzeugt:

```
angle0 = twoPi * i / segments;  
angle1 = twoPi * (i + 1) / segments;  
ring = (cos(angle) * radius, sin(angle) * radius, 0);
```

Der Mantel bekommt radiale Normalen, Deckel und Boden bekommen konstante Normalen.

Kugel

Die Kugel ist eine UV-Sphere aus Stacks und Slices:

Kugelkoordinaten:

```
x = radius * cos(theta) * sin(phi);  
y = radius * sin(theta) * sin(phi);  
z = radius * cos(phi);
```

Die Normale ist `Normalize(position)`, weil die Kugel im Ursprung liegt.

Winding

Da `glFrontFace(GL_CCW)` und `GL_CULL_FACE` aktiv sind, muss die Dreiecksreihenfolge von aussen betrachtet gegen den Uhrzeigersinn sein. Sonst werden sichtbare Flaechen als Rueckseiten behandelt.

Glossar

- **Primitive:** Einfache Grundkoerper.
- **Dreiecksliste:** Vertexliste, in der je drei Vertices ein Dreieck bilden.
- **Normale:** Richtung senkrecht zur Oberflaeche.
- **Stack:** Kugel-Unterteilung von Pol zu Pol.
- **Slice:** Kugel-Unterteilung rundherum.
- **Theta:** Rundum-Winkel von 0 bis 2 pi.
- **Phi:** Polarwinkel von 0 bis pi.
- **Winding:** Reihenfolge der Dreieckspunkte.
- **CCW:** Counter-clockwise, gegen den Uhrzeigersinn.
- **Face Culling:** Entfernen von Rueckseiten.

▼ Aufgabe 3.3 - Rendern der Geometrie

Ziel

Die erzeugten Primitive werden auf die GPU geladen und in der Szene gerendert. Quader, Zylinder, Kugel und Lichtmarker besitzen jeweils eigene VBOs, aber dasselbe VAO-Layout.

Mehrere VBOs, ein VAO

Vor jedem Drawcall wird Binding 0 auf den passenden VBO gesetzt:

```
glVertexArrayVertexBuffer(VAO, 0, meshes[meshIndex].vbo, 0, sizeof(Vertex));
```

Das Format bleibt gleich:

- Location 0: Position
- Location 1: Normale
- Location 2: Farbe

Glossar

- **VBO**: Speicher fuer Vertexdaten.
- **VAO**: Layoutbeschreibung fuer Vertexattribute.
- **Bindingpunkt**: Slot im VAO, an den ein VBO gebunden wird.
- **Drawcall**: Zeichenbefehl.
- **Model-Matrix**: Positioniert, skaliert oder rotiert ein Objekt.

▼ Aufgabe 3.4 - Debugging der Normalenvektoren

Ziel

Es soll geprueft werden, ob Normalen richtig erzeugt und richtig transformiert werden.

Methode 1: Normalen als Farben

Normalen liegen im Bereich `[-1, 1]`, Farben im Bereich `[0, 1]`. Deshalb wird umgerechnet:

```
vec3 normalColor = normalize(in_Normal) * 0.5 + 0.5;
outColor = vec4(normalColor, 1.0);
```

Methode 2: Normalenlinien

Aus Stichproben der Vertices werden Linien erzeugt:

```
endPosition = position + Normalize(normal) * length;
```

Glossar

- **Normalen-Debugging**: Sichtbarmachen von Normalen.
- **Normalenlinie**: Linie in Normalenrichtung.
- **GL_LINES**: OpenGL zeichnet je zwei Vertices als Linie.
- **Remapping**: Umrechnung von `[-1,1]` nach `[0,1]`.
- **ImGui**: UI-Bibliothek fuer Debugfenster.

▼ Aufgabe 3.5 - Beleuchtung

Ziel

Die Szene wird mit einer Punktlichtquelle und Lambert-Beleuchtung gerendert.

Lambert-Modell

```
vec3 lightDir = normalize(u_LightViewPos - in_ViewSpacePos);
float diffuse = max(dot(normal, lightDir), 0.0);
vec3 ambient = 0.18 * in_Color;
vec3 litColor = ambient + diffuse * in_Color;
```

Koordinatenraum

Fragmentposition, Normale und Lichtposition werden im View-Space benutzt. Das ist wichtig, weil Beleuchtungsrechnung nur korrekt ist, wenn alle beteiligten Vektoren im selben Raum liegen.

Normal-Matrix

Bei nicht-uniformer Skalierung dürfen Normalen nicht einfach mit der Model-View-Matrix transformiert werden. Der Vertexshader nutzt:

```
mat3 normalMat = transpose(inverse(mat3(modelViewMat)));
out_Normal = normalize(normalMat * in_Normal);
```

Glossar

- **Lambert:** Diffuses Beleuchtungsmodell.
- **Punktlichtquelle:** Licht mit Position im Raum.
- **Dot Product:** Skalarprodukt; misst Winkelnahe zweier Richtungen.
- **Ambient:** Konstanter Grundlichtanteil.
- **View-Space:** Koordinatensystem aus Kamerasicht.
- **Normal-Matrix:** Inverse transponierte Matrix fuer Normalen.

▼ Aufgabe 3.6 - Bewegung der Kamera

Ziel

Die Kamera wird per Tastatur entlang ihrer eigenen Achsen bewegt und rotiert. Die Bewegung ist framerate-unabhaengig.

Steuerung

- `W/S` : vor/zurueck
- `A/D` : links/rechts
- `Q/E` : runter/hoch
- Pfeile: Yaw/Pitch
- `R/F` : Roll
- `Shift` : schneller

Delta-Time

`moveStep = moveSpeed * deltaSeconds;`

`rotateStep = rotateSpeed * deltaSeconds;`

Damit haengt die Geschwindigkeit von Sekunden ab, nicht von Frames.

Glossar

- **Kameraachsen:** Forward, Right, Up.
- **Yaw:** Links/rechts drehen.
- **Pitch:** Hoch/runter drehen.
- **Roll:** Um die Blickrichtung drehen.
- **Delta-Time:** Zeit seit letztem Frame.
- **SDL Scancode:** Physische Taste in SDL.

▼ Aufgabe 3.7 - Matrizen-Stack und animierte Szene

Ziel

Eine animierte Kran-Szene wird hierarchisch mit einem MatrixStack aufgebaut.

Hierarchie

MatrixStack

- `Push` : aktuelle Matrix kopieren
- `Pop` : zur Elternmatrix zurueck
- `Last` : aktuelle Matrix
- `Translate` , `Rotate` , `Scale` : Transformationen anhängen

Animation

`craneYaw = sinf(elapsedSeconds * 0.7f) * 35.0f;`

Dadurch schwenkt der Ausleger periodisch.

Glossar

- **MatrixStack:** Stapel von Transformationsmatrizen.

- **Hierarchie:** Kindobjekte erben Elterntransformationen.
- **Translate:** Verschieben.
- **Rotate:** Drehen.
- **Scale:** Skalieren.
- **Animation:** Zeitabhaengige Veraenderung.
- **Stack-Tiefe:** Anzahl verschachtelter Matrizen.

▼ Aufgabe 3.8 - Ship It

Ziel

Mit `shipit.bat` wird ein Release-Build erzeugt und zusammen mit DLLs und Assets gepackt.

Befehl

```
shipit.bat tasks_src\task03\
```

Das Ergebnis liegt unter:

```
release_build\task03-Win-x64.zip
```

Inhalt

- `task03.exe`
- `SDL3.dll`
- `physfs.dll`
- `assets/`

Glossar

- **Release-Build:** Optimierte Buildvariante fuer Abgabe.
- **ZIP:** Gepacktes Archiv.
- **DLL:** Dynamische Windows-Bibliothek.
- **EXE:** Ausfuehrbare Windows-Datei.
- **Target:** Konkretes CMake-Bauziel.

▼ Aufgabe 4.0 - Setup Texturen

Ziel

Aufgabe 4 startet mit `tasks_src/task04/task04.cpp`. Das Programm laedt Shader, ein Bild und rendert zuerst ein rotes Quad.

Build und Run

```
cmake -B build -S .  
cmake --build build --target task04 --config Debug  
.\build\Debug\task04.exe assets\
```

Glossar

- **Textur:** Bilddaten, die auf Geometrie gelegt werden.
- **Quad:** Viereck aus zwei Dreiecken.
- **Asset-Pfad:** Pfad zu Shadern, Texturen und Modellen.
- **SDL:** Bibliothek fuer Fenster und Eingaben.

▼ Aufgabe 4.1 - Texturkoordinaten fuer ein Quad

Ziel

Die vier Quad-Vertices bekommen UV-Koordinaten. Position beschreibt den Ort im Raum, UV beschreibt den Ort auf dem Bild.

Aktueller Code

```
links unten -> uv = (0, 1)  
rechts unten -> uv = (1, 1)  
rechts oben -> uv = (1, 0)  
links oben -> uv = (0, 0)
```

Die `v`-Werte sind gedreht, damit das Bild nicht auf dem Kopf steht.

Diagramm

Glossar

- **UV:** Texturkoordinate. `u` horizontal, `v` vertikal.
- **Texel:** Ein Bildpunkt einer Textur.
- **Mapping:** Zuordnung von Geometrie zu Bildpositionen.
- **Quad:** Viereck aus zwei Dreiecken.
- **Bildursprung:** Stelle, an der Bildkoordinaten beginnen, oft oben links.

▼ Aufgabe 4.2 - OpenGL-Textur auf der GPU erstellen

Ziel

Das geladene Bild wird als OpenGL-Textur auf die GPU kopiert.

Datenfluss

Relevanter Code

```

GLuint textureHandle = 0;
glCreateTextures(GL_TEXTURE_2D, 1, &textureHandle);
glTextureParameteri(textureHandle, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTextureParameteri(textureHandle, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
glTextureParameteri(textureHandle, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTextureParameteri(textureHandle, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTextureStorage2D(textureHandle, 1, GL_RGBA8, image.GetWidth(),
image.GetHeight());
glTextureSubImage2D(textureHandle, 0, 0, 0, image.GetWidth(),
image.GetHeight(), GL_RGBA, GL_UNSIGNED_BYTE, image.Data());

```

Glossar

- **Texture Object:** OpenGL-Objekt fuer Texturdaten.
- **RGBA8:** 8 Bit pro Rot-, Gruen-, Blau- und Alpha-Kanal.
- **Wrap Mode:** Verhalten ausserhalb UV `[0,1]` .
- **GL_CLAMP_TO_EDGE:** Randpixel werden weiterverwendet.
- **Filter:** Auswahl/Mischung von Texeln.
- **GL_NEAREST:** Naechster Texel wird genommen.
- **stb_image:** Bibliothek zum Laden von Bilddateien.

▼ Aufgabe 4.3 - Texturkoordinaten im Shader zuganglich machen

Ziel

Die UV-Koordinaten aus dem VBO werden ueber das VAO an den Vertexshader und weiter an den Fragmentshader gegeben.

Datenfluss

Shadercode

Vertexshader:

```

layout(location = 3) in vec3 in_UV;
layout(location = 3) out vec3 out_UV;
out_UV = in_UV;

```

Fragmentshader:

```

layout(location = 3) in vec3 in_UV;

```

Glossar

- **Shaderattribute:** Eingaben des Vertexshaders.
- **Location 3:** Slot fuer UV-Koordinaten.
- **Interpolation:** Zwischenwerte ueber ein Dreieck.
- **Rasterizer:** Pipeline-Stufe, die Dreiecke in Fragmente zerlegt.

- **Swizzle:** Zugriff wie `.st` auf Vektorkomponenten.

▼ Aufgabe 4.4 - Textur im Fragmentshader sampeln

Ziel

Der Fragmentshader liest mit den interpolierten UV-Koordinaten die passende Farbe aus der Textur.

Code

```
layout(binding = 0) uniform sampler2D u_Texture;

void main()
{
    vec4 texColor = texture(u_Texture, in_UV.st);
    outColor = texColor;
}
```

CPU-seitig wird die Textur gebunden:

```
glBindTextureUnit(0, textureHandle);
```

Diagramm

Glossar

- **Sampler:** Shader-Zugriff auf eine Textur.
- **sampler2D:** Sampler fuer 2D-Texturen.
- **Texture Unit:** Bindslot fuer Texturen.
- **Binding 0:** Slot, an dem `u_Texture` die Textur erwartet.
- **Sampling:** Auslesen einer Texturfarbe an UV-Koordinaten.

▼ Aufgabe 4.5 - Texturkoordinaten fuer die Kugel

Ziel

Die Kugel aus Aufgabe 3 wird so erweitert, dass sie UV-Koordinaten besitzt. Dadurch kann eine Weltkarte auf die Kugel gelegt werden.

Weltkarte

Die Datei liegt hier:

`assets/textures/world.200404.3x5400x2700.jpg`

Sie hat ein 2:1-Seitenverhaeltnis. Das passt zu einer equirectangular world map: links/rechts ist einmal um die Erde herum, oben/unten ist Nordpol bis Suedpol.

Kugelparametrisierung

Die Kugel wird ueber zwei Winkel erzeugt:

- `theta` : Rundum-Winkel von `0` bis `2*pi` .
- `phi` : Polarwinkel von `0` bis `pi` .

3D-Position:

```
x = radius * cos(theta) * sin(phi);  
y = radius * sin(theta) * sin(phi);  
z = radius * cos(phi);
```

UV-Koordinaten:

```
u = theta / (2*pi);  
v = phi / pi;
```

Aufbau aus Stacks und Slices

Polfaelle

Am Nord- und Suedpol fallen mehrere Ringpunkte zusammen. Deshalb wird im ersten und letzten Stack jeweils nur ein Dreieck pro Slice erzeugt. In der Mitte werden zwei Dreiecke pro Gitterzelle erzeugt.

Drawcall

Die Kugel wird als fertige Dreiecksliste erzeugt, deshalb wird kein EBO gebraucht:

```
glDrawArrays(GL_TRIANGLES, 0, (GLsizei)sphereVertices.size());
```

Typische Fehler

- Kugel unsichtbar: Winding passt nicht zu `glFrontFace(GL_CCW)` .
- Textur kopfueber: `v` als `1.0f - phi / pi` testen.
- Naht sichtbar: normal bei `u = 0` / `u = 1` .
- Verzerrung an Polen: normal bei equirectangular Mapping.

Glossar

- **UV-Sphere**: Kugel aus Laengen- und Breitengraden.
- **Equirectangular**: Rechteckige Weltkarte fuer Kugelmapping.
- **Theta**: Rundum-Winkel.
- **Phi**: Winkel von Nord nach Sued.
- **Stack**: horizontale Kugelunterteilung.
- **Slice**: vertikale/rundum Kugelunterteilung.
- **Naht**: Stelle, an der `u = 0` und `u = 1` zusammentreffen.

- **Polverzerrung:** Verzerrung nahe Nord- und Suedpol.
- **Winding:** Reihenfolge der Dreieckspunkte.

▼ Aufgabe 4.6 - Rotation der Kugel

Ziel

Die Kugel soll mit Pfeiltasten rotiert werden. Diese Aufgabe ist nach dem Rueckbau der experimentellen Blickwinkel-Rotation noch offen bzw. sollte wieder als normale, einfache Aufgabenloesung umgesetzt werden.

Einfache Zielidee

Man speichert zwei Winkel:

```
float sphereYaw = 0.0f;
float spherePitch = 0.0f;
```

Die Pfeiltasten veraendern diese Winkel mit Delta-Time. Danach wird die Model-Matrix gebaut:

```
modelMat = Rotate(RAMEN_WORLD_UP, sphereYaw) * Rotate(RAMEN_WORLD_RIGHT,
spherePitch);
```

Diagramm

Glossar

- **Yaw:** Links/rechts-Drehung.
- **Pitch:** Hoch/runter-Drehung.
- **Model-Matrix:** Objekttransformation.
- **Delta-Time:** Zeit seit dem letzten Frame.
- **Framerate-unabhaengig:** Gleich schnell trotz unterschiedlicher FPS.